AGENTIC AI WORKSHOP · DAY 2 OF 10

Core single-agent patterns

Hand-build a ReAct agent from raw API calls, then add planning, a critic and guardrails. No framework yet.

You build: **Agent loop** · about 2.7 h of material

Contents

1. Build the loop by hand, once — Why this matters · 5 min
2. The ReAct loop — Theory · 12 min
3. Tools are contracts — Theory · 10 min
4. Lab 1: Build the ReAct agent — Lab · beginner · 40 min
5. Plan-and-Execute — Theory · 10 min
6. Lab 2: Plan, then execute — Lab · intermediate · 25 min
7. Reflection & Reflexion — Theory · 10 min
8. Lab 3: Catch the currency bug — Lab · intermediate · 20 min
9. Guardrails: stop runaway agents — Theory · 8 min
10. Lab 4: Break it on purpose — Lab · intermediate · 15 min
11. Check yourself — Quiz · 5 min
12. Day 2 recap — Recap · 3 min
13. Quiz answer key

WHY THIS MATTERS 5 MIN

Build the loop by hand, once

Tomorrow you'll use a framework. Today you'll write the ~100 lines it hides, so it never feels like magic.

Frameworks are great until something goes wrong. Then you debug the loop **you understand**. Today Meridian learns to answer **"Is ACME over its processing limit?"** with real tool calls, and to survive mistakes along the way.

The data: three fictional merchants, each with a monthly processing limit and this month's settlement batches in different currencies.

Merchant	Limit	This month	Verdict
ACME	9.5M USD	4.2M EUR + 3.1M USD + 2.0M EUR	**OVER_LIMIT** (9.92M, 104.4%)
BOLT	5M GBP	1.5M USD + 2.0M GBP	OK (63%)
CRUX	800M INR	650M INR + 2M USD	**OVER_LIMIT** (820M, 102.5%)

By the end of today you can

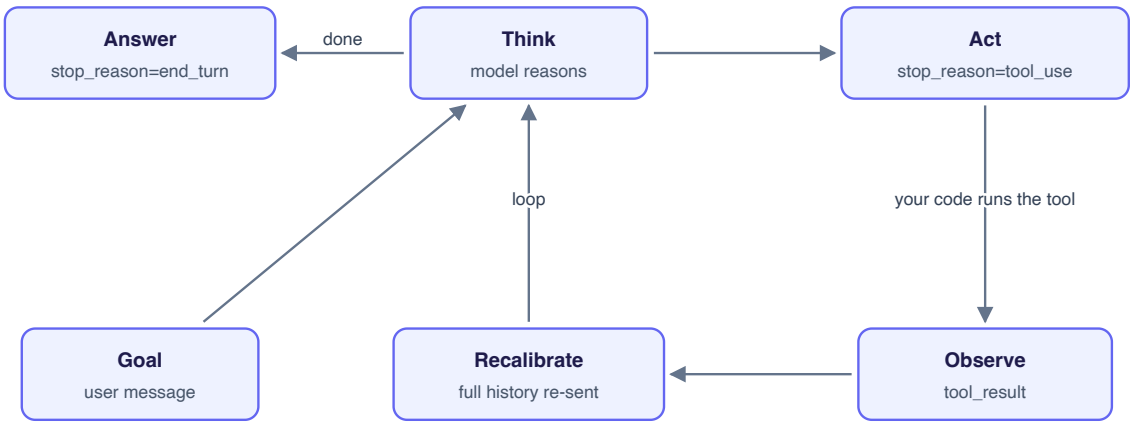
- Write a ReAct loop on the raw Messages API
- Design tool schemas with pydantic so the model calls tools correctly
- Separate planning from execution with a plan your code owns
- Add a critic, and know when it helps vs when it just burns tokens
- Stop runaway agents with three guardrails

THEORY 12 MIN

The ReAct loop

Reason + Act: think → act → observe → recalibrate, until done.

The ReAct loop and the API fields behind it



1. The goal is the first user message: 'Is ACME over its processing limit?'
2. The model thinks, then asks for a tool: the response has stop_reason == "tool_use" and a tool_use block.
3. YOUR code runs lookup_merchant('ACME') and sends back a tool_result. The model never executes anything itself.
4. Next call: the whole conversation is re-sent. Seeing EUR batches, the model asks for get_fx_rate, then the calculator.
5. When it has enough, stop_reason == "end_turn": OVER_LIMIT, 104.4%. Your loop ends.

Key points

- An agent is a while loop. The model only decides what goes into it.
- The model never runs a tool. It requests one, and YOUR code decides whether to run it. That's where safety checks go.
- State is just the message list. Always append the model's full reply (history is append-only).
- Several tool calls can come back in one turn: return ALL their results in ONE user message.

THEORY 10 MIN

Tools are contracts

The model sees only a tool's name, description and JSON schema. That's the whole manual it gets.

We write each tool's input as a **pydantic model**: the docstring becomes the description and the fields become the schema.

Pydantic model → tool definition

```
class FxInput(BaseModel):
    """Get the FX rate to convert 1 unit of `base` into `quote`. Use it for EVERY currency conversion."""
    base: Literal["USD", "EUR", "GBP", "INR"]
    quote: Literal["USD", "EUR", "GBP", "INR"]
```

```
# ...
TOOLS = [
    {"name": "lookup_merchant", "description": MerchantInput.__doc__, "input_schema":
MerchantInput.model_json_schema()},
    {"name": "get_fx_rate", "description": FxInput.__doc__, "input_schema": FxInput.model_json_schema()},
    {"name": "calculator", "description": CalcInput.__doc__, "input_schema":
CalcInput.model_json_schema()},
]
```

Rule	Bad	Good
Use enums, not free text	`currency: str`	`Literal["USD", "EUR", ...]`
Say **when** to use it	"fx tool"	"Use it for EVERY currency conversion"
One job per tool	`do_payments(action, ...)`	`get_fx_rate`, `lookup_merchant`
Errors are messages	crash	return `is_error=True` + a hint ("valid ids: ACME, BOLT, CRUX")
Safe code	`eval(expression)`	a tiny `ast` evaluator: numbers and + - * / only

Key points

- A bad tool description causes bad behaviour, and no framework fixes that.
- Return errors to the model so it can recover, e.g. from an unknown merchant id.

LAB BEGINNER 40 MIN

Lab 1: Build the ReAct agent

One file, four sections: data → tool schemas → tool code → the loop.

Goal. Meridian answers "Is ACME over its processing limit?" by calling tools, with no framework.

1. Set up Day 2

RUN

```
CD day2-single-agent-patterns/code
CP .env.example .ENV              # ADD YOUR OPENROUTER KEY
pip install -r requirements.txt
```

2. Read the tool schemas

Pydantic models → `model_json_schema()` → the `TOOLS` list the model reads.

lab1_react_agent.py: tool schemas

```python
# ---------------- 2. Tool schemas (pydantic -> JSON schema the model reads) ----------------
class MerchantInput(BaseModel):
    """Look up a merchant: monthly processing limit and this month's settlement batches (amount, currency)."""
    merchant_id: str = Field(description="e.g. ACME")


class FxInput(BaseModel):
    """Get the FX rate to convert 1 unit of `base` into `quote`. Use it for EVERY currency conversion."""
    base: Literal["USD", "EUR", "GBP", "INR"]
    quote: Literal["USD", "EUR", "GBP", "INR"]


class CalcInput(BaseModel):
    """Calculate an arithmetic expression, e.g. '(4200000+2000000)*1.10'. Use it for ALL maths."""
    expression: str
```

3. Read the loop: the heart of the agent

Find THINK, ACT and OBSERVE in the comments.

lab1_react_agent.py: the loop

```python
def run_agent(question: str, system: str = SYSTEM, tools: list = TOOLS) -> str:
    messages = [{"role": "user", "content": question}]

    for turn in range(1, MAX_TURNS + 1):
        response = client.messages.create(model=MODEL, max_tokens=4000, system=system,
                                          tools=tools, messages=messages)
        messages.append({"role": "assistant", "content": response.content})   # keep the FULL reply

        if response.stop_reason != "tool_use":                                # THINK done -> answer
            return "".join(b.text for b in response.content if b.type == "text")

        results = []
        for block in response.content:
            if block.type == "tool_use":                                       # ACT
                output, is_error = run_tool(block.name, block.input)
```

```
                print(f"[turn {turn}] {block.name}({block.input}) -> {output}")  # OBSERVE
                results.append({"type": "tool_result", "tool_use_id": block.id,
                                "content": output, "is_error": is_error})
        messages.append({"role": "user", "content": results})                  # all results in ONE
message

    return f"Stopped after {MAX_TURNS} turns."
```

4. Run it

RUN `python lab1_react_agent.py`

5. Try the inverse-rate trap

BOLT's limit is in GBP, but the table only has GBP→USD. Watch the tool invert the rate.

RUN `python lab1_react_agent.py "IS BOLT OVER ITS PROCESSING LIMIT?"`

6. Try an unknown merchant

The tool returns an error with a hint, and the model recovers.

RUN `python lab1_react_agent.py "IS ZETA OVER ITS PROCESSING LIMIT?"`

Expected result. `[turn 1] lookup_merchant(...)`, `[turn 2] get_fx_rate(...)`, then the calculator, then **OVER_LIMIT, about 104.4%**.

Full file

`lab1_react_agent.py`

```
"""Day 2 – Lab 1: a ReAct agent from scratch. No framework: just a loop.

Run:  python lab1_react_agent.py
      python lab1_react_agent.py "Is BOLT over its processing limit?"
"""
import ast
import json
import operator
import sys
from typing import Literal

from pydantic import BaseModel, Field

from llm import MODEL, client   # Claude via OpenRouter (see llm.py)

# ---------------- 1. Data (fictional) ----------------
MERCHANTS = {
    "ACME": {"limit": 9_500_000, "limit_currency": "USD",
             "batches": [(4_200_000, "EUR"), (3_100_000, "USD"), (2_000_000, "EUR")]},
    "BOLT": {"limit": 5_000_000, "limit_currency": "GBP",
             "batches": [(1_500_000, "USD"), (2_000_000, "GBP")]},
    "CRUX": {"limit": 800_000_000, "limit_currency": "INR",
             "batches": [(650_000_000, "INR"), (2_000_000, "USD")]},
}
FX = {("EUR", "USD"): 1.10, ("GBP", "USD"): 1.30, ("USD", "INR"): 85.0}
```

```python
# ---------------- 2. Tool schemas (pydantic -> JSON schema the model reads) ----------------
class MerchantInput(BaseModel):
    """Look up a merchant: monthly processing limit and this month's settlement batches (amount,
currency)."""
    merchant_id: str = Field(description="e.g. ACME")


class FxInput(BaseModel):
    """Get the FX rate to convert 1 unit of `base` into `quote`. Use it for EVERY currency conversion."""
    base: Literal["USD", "EUR", "GBP", "INR"]
    quote: Literal["USD", "EUR", "GBP", "INR"]


class CalcInput(BaseModel):
    """Calculate an arithmetic expression, e.g. '(4200000+2000000)*1.10'. Use it for ALL maths."""
    expression: str


# ---------------- 3. Tool implementations (plain Python) ----------------
def lookup_merchant(merchant_id: str) -> dict:
    if merchant_id.upper() not in MERCHANTS:
        raise ValueError(f"unknown merchant. Valid ids: {list(MERCHANTS)}")
    return MERCHANTS[merchant_id.upper()]


def get_fx_rate(base: str, quote: str) -> float:
    if base == quote:
        return 1.0
    if (base, quote) in FX:
        return FX[(base, quote)]
    if (quote, base) in FX:
        return round(1 / FX[(quote, base)], 6)       # inverse rate
    raise ValueError(f"no rate for {base}/{quote}, convert via USD")


OPS = {ast.Add: operator.add, ast.Sub: operator.sub, ast.Mult: operator.mul, ast.Div: operator.truediv}


def calculator(expression: str) -> float:
    """Safe maths: numbers and + - * / only. Never use eval()!"""
    def ev(node):
        if isinstance(node, ast.Constant):
            return node.value
        if isinstance(node, ast.BinOp) and type(node.op) in OPS:
            return OPS[type(node.op)](ev(node.left), ev(node.right))
        raise ValueError("only numbers and + - * / are allowed")
    return round(ev(ast.parse(expression, mode="eval").body), 4)


TOOLS = [
    {"name": "lookup_merchant", "description": MerchantInput.__doc__, "input_schema":
MerchantInput.model_json_schema()},
    {"name": "get_fx_rate", "description": FxInput.__doc__, "input_schema": FxInput.model_json_schema()},
    {"name": "calculator", "description": CalcInput.__doc__, "input_schema":
CalcInput.model_json_schema()},
]
FUNCTIONS = {"lookup_merchant": lookup_merchant, "get_fx_rate": get_fx_rate, "calculator": calculator}


def run_tool(name: str, args: dict) -> tuple[str, bool]:
    """Run a tool. Errors are returned to the model (is_error=True) so it can recover."""
    try:
        return json.dumps(FUNCTIONS[name](**args)), False
    except Exception as e:
```

```
        return f"Error: {e}", True


# ---------------- 4. The ReAct loop ----------------
SYSTEM = """You are a merchant-risk analyst at Northstar Pay (a payment processor).
Use ONLY the tools. Convert every amount into the limit's currency before comparing.
Finish with one line: OVER_LIMIT / NEAR_LIMIT (above 90%) / OK, and the utilisation %."""
MAX_TURNS = 10


def run_agent(question: str, system: str = SYSTEM, tools: list = TOOLS) -> str:
    messages = [{"role": "user", "content": question}]

    for turn in range(1, MAX_TURNS + 1):
        response = client.messages.create(model=MODEL, max_tokens=4000, system=system,
                                          tools=tools, messages=messages)
        messages.append({"role": "assistant", "content": response.content})   # keep the FULL reply

        if response.stop_reason != "tool_use":                                # THINK done -> answer
            return "".join(b.text for b in response.content if b.type == "text")

        results = []
        for block in response.content:
            if block.type == "tool_use":                                       # ACT
                output, is_error = run_tool(block.name, block.input)
                print(f"[turn {turn}] {block.name}({block.input}) -> {output}")  # OBSERVE
                results.append({"type": "tool_result", "tool_use_id": block.id,
                                "content": output, "is_error": is_error})
        messages.append({"role": "user", "content": results})                  # all results in ONE
message

    return f"Stopped after {MAX_TURNS} turns."


if __name__ == "__main__":
    question = " ".join(sys.argv[1:]) or "Is ACME over its processing limit?"
    print("\nANSWER:", run_agent(question))
```

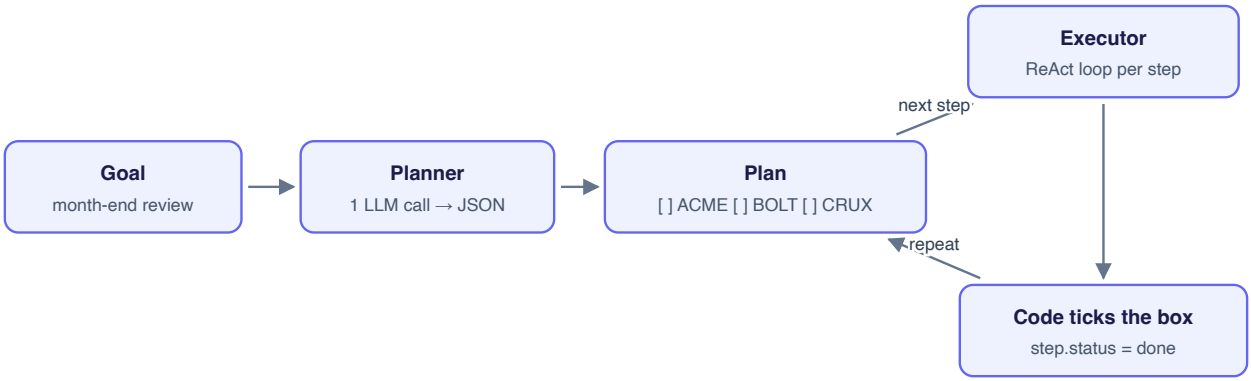
THEORY 10 MIN

Plan-and-Execute

For multi-step tasks, separate "what's the plan?" from "do the next step".

Ask a pure ReAct agent for a **month-end review of three merchants** and it tends to drift: it forgets CRUX, re-fetches rates or finishes early. Real analysts work from a **checklist**, and so should the agent.

Plan-and-Execute: the plan is state your code owns



1. One planning call turns the goal into a checklist (JSON, validated with pydantic).
2. The Plan object lives in YOUR code: it's inspectable, auditable and resumable.
3. For the current step only, the executor runs the ReAct loop from Lab 1.
4. Code (not the model) marks the step done and records its result.
5. Repeat until every box is ticked. No more forgetting CRUX or finishing early.

	ReAct	Plan-and-Execute
Best for	short, exploratory questions	checklist-shaped work: reviews, reconciliations, reports
Audit	read the transcript	the plan itself is the audit record
Watch out for	drift, finishing early	a stale plan, so add a re-plan step

LAB INTERMEDIATE 25 MIN

Lab 2: Plan, then execute

Goal. A planner writes a checklist; your code runs the Lab 1 agent once per step and ticks the boxes.

1. Read the planner

It asks for JSON and validates it with pydantic, so you get a typed `Plan`, not text.

lab2_plan_and_execute.py: planner

```
# ---------- 1. PLAN: ask for JSON, validate it with pydantic ----------
PLANNER_PROMPT = f"""Break this goal into a short checklist. One step per merchant, plus one final
summary step.
Reply with JSON only, like: {{"goal": "...", "steps": [{{"id": 1, "description": "..."}}]}}

Goal: {GOAL}"""

reply = client.messages.create(model=MODEL, max_tokens=2000, messages=[{"role": "user", "content":
PLANNER_PROMPT}])
text = "".join(b.text for b in reply.content if b.type == "text")
plan = Plan.model_validate(json.loads(text[text.find("{"): text.rfind("}") + 1]))
```

2. Read the executor

Notice who marks progress: `step.status = "done"` is **code**, not the model.

lab2_plan_and_execute.py: executor

```
# ---------- 2. EXECUTE: code owns the plan and ticks the boxes ----------
for step in plan.steps:
    print(f"\n>>> step {step.id}: {step.description}")
    done_so_far = "\n".join(f"- {s.description}: {s.result}" for s in plan.steps if s.status ==
"done")
    step.result = run_agent(f"Overall goal: {plan.goal}\n\nResults so far:\n{done_so_far or
'(none)'}\n\n"
                            f"Do ONLY this step now: {step.description}")
    step.status = "done"                        # the CODE marks progress, not the model

print("\nFINAL PLAN\n" + show(plan))
print("\nSUMMARY\n" + plan.steps[-1].result)
```

3. Run it

RUN `python lab2_plan_and_execute.py`

4. Compare with Lab 1

Ask the Lab 1 agent the same month-end goal in one go. Which one is more reliable, and which one is easier to audit?

Expected result. A plan with one step per merchant plus a summary; each step gets a result; a final summary table.

Full file

lab2_plan_and_execute.py

```
"""Day 2 - Lab 2: separate PLANNING from EXECUTION.

1. The planner (one LLM call) writes a checklist as JSON -> a typed Plan object.
2. CODE walks the checklist. For each step it runs the Lab 1 agent, then marks the step done.

Run:  python lab2_plan_and_execute.py
"""
import json

from pydantic import BaseModel

from lab1_react_agent import MODEL, client, run_agent   # reuse Lab 1's client, tools and loop


class Step(BaseModel):
    id: int
    description: str
    status: str = "pending"        # pending -> done
    result: str = ""


class Plan(BaseModel):
    goal: str
    steps: list[Step]


GOAL = ("Month-end review: for ACME, BOLT and CRUX give MTD volume in the limit currency, utilisation % "
        "and verdict. Then a summary table.")

# ---------- 1. PLAN: ask for JSON, validate it with pydantic ----------
PLANNER_PROMPT = f"""Break this goal into a short checklist. One step per merchant, plus one final
summary step.
Reply with JSON only, like: {{"goal": "...", "steps": [{{"id": 1, "description": "..."}}]}}

Goal: {GOAL}"""

reply = client.messages.create(model=MODEL, max_tokens=2000, messages=[{"role": "user", "content":
PLANNER_PROMPT}])
text = "".join(b.text for b in reply.content if b.type == "text")
plan = Plan.model_validate(json.loads(text[text.find("{"): text.rfind("}") + 1]))


def show(plan: Plan) -> str:
    return "\n".join(f"[{'x' if s.status == 'done' else ' '}] {s.id}. {s.description}  {s.result[:70]}"
                     for s in plan.steps)


print("PLAN\n" + show(plan))

# ---------- 2. EXECUTE: code owns the plan and ticks the boxes ----------
for step in plan.steps:
    print(f"\n>>> step {step.id}: {step.description}")
    done_so_far = "\n".join(f"- {s.description}: {s.result}" for s in plan.steps if s.status == "done")
    step.result = run_agent(f"Overall goal: {plan.goal}\n\nResults so far:\n{done_so_far or
'(none)'}\n\n"
                            f"Do ONLY this step now: {step.description}")
    step.status = "done"                        # the CODE marks progress, not the model

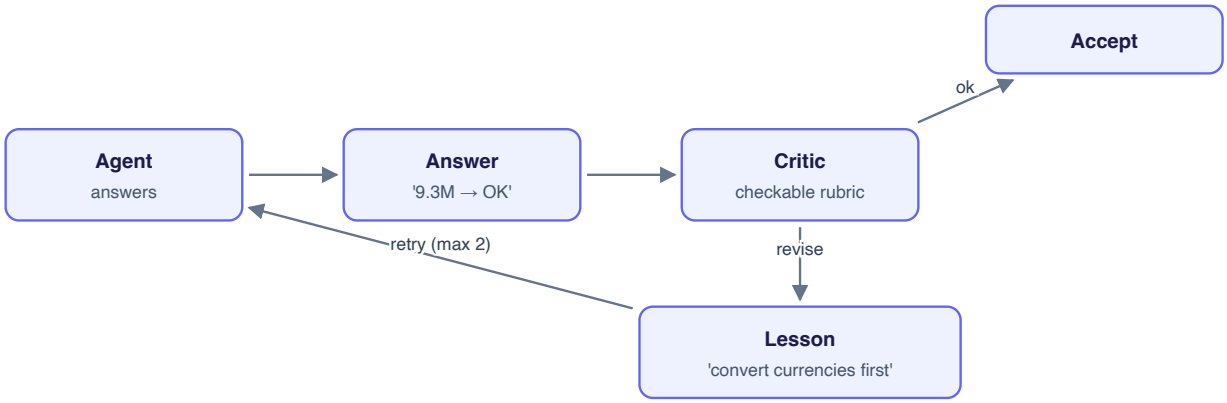
print("\nFINAL PLAN\n" + show(plan))
print("\nSUMMARY\n" + plan.steps[-1].result)
```

THEORY 10 MIN

Reflection & Reflexion

Let a critic check the work, but only when it has something real to check against.

Reflexion: critique, learn, retry (with a cap)



1. Without an FX tool, the agent adds EUR and USD together: 9.3M 'USD', so 'OK'. Wrong.
2. A critic checks the answer against a short, checkable rubric AND the raw facts.
3. It finds the bug and writes a reusable lesson: 'convert to the limit currency before adding'.
4. The agent retries with the lesson (and the FX tool). Cap the retries: critics can loop forever.
5. Now 9.92M USD → OVER_LIMIT. Critics help when there's a checkable signal, and burn tokens when there isn't.

It helps when there's a **checkable** rule (same currency? all batches included? verdict matches the numbers?) and mistakes are expensive.

It burns tokens when there's no real signal ("is this good?" → "yes, but could be more detailed" forever), when you run it on every trivial step, or when retries aren't capped.

Key points

- Give the critic the facts, not just the answer.
- Always cap the retries (we use 2).
- Measure: did the critic change answers, and were the changes corrections?

LAB INTERMEDIATE 20 MIN

Lab 3: Catch the currency bug

Goal. Plant a bug (no FX tool), watch the critic catch it, and see the agent fix it using the critic's lesson.

1. Read the planted weakness

The agent first runs **without** `get_fx_rate`, so it will add EUR and USD together.

lab3_reflexion.py: the weak agent

```
question = "Is ACME over its processing limit?"
tools_without_fx = [t for t in TOOLS if t["name"] != "get_fx_rate"]      # the planted weakness
weak_system = ("You are a merchant-risk analyst. Use the tools. Show your numbers, "
               "then finish with OVER_LIMIT / NEAR_LIMIT / OK.")
```

2. Read the critic

A short, **checkable** rubric, plus the merchant's raw facts.

lab3_reflexion.py: critic

```
CRITIC_RUBRIC = """You check a merchant-risk answer. Rules:
1. All amounts must be converted to the limit's currency before adding (an FX rate must be used).
2. The verdict must match the numbers (over 100% = OVER_LIMIT, over 90% = NEAR_LIMIT, else OK).
Reply with JSON only: {"ok": true/false, "issue": "...", "lesson": "a short rule to remember"}"""


def critique(question: str, answer: str, facts: dict) -> dict:
    # A critic is only useful with something CHECKABLE: give it the raw facts, not just the answer.
    content = f"Question: {question}\n\nFacts from the merchant file: {facts}\n\nAnswer to check:
{answer}"
    r = client.messages.create(model=MODEL, max_tokens=1000, system=CRITIC_RUBRIC,
                               messages=[{"role": "user", "content": content}])
    text = "".join(b.text for b in r.content if b.type == "text")
    return json.loads(text[text.find("{"): text.rfind("}") + 1])
```

3. Run it

RUN `python lab3_reflexion.py`

4. Discuss

Did the critic catch it? With Claude it should. With a tiny local model it may not: a critic is only as good as its model **and** its evidence.

Expected result. FIRST ANSWER is wrong (OK / 9.3M) → CRITIC finds the currency issue → REVISED ANSWER: **OVER_LIMIT, 9.92M USD**.

Full file

lab3_reflexion.py

```
"""Day 2 - Lab 3: Reflexion. A critic checks the answer; if it finds a problem, the agent retries
with the critic's lesson. Max 2 retries.

We first make the bug happen on purpose: the agent gets NO FX tool, so it will add EUR and USD
```

```
together. The critic should catch it.

Run:  python lab3_reflexion.py
"""
import json

from lab1_react_agent import MERCHANTS, MODEL, TOOLS, client, run_agent

CRITIC_RUBRIC = """You check a merchant-risk answer. Rules:
1. All amounts must be converted to the limit's currency before adding (an FX rate must be used).
2. The verdict must match the numbers (over 100% = OVER_LIMIT, over 90% = NEAR_LIMIT, else OK).
Reply with JSON only: {"ok": true/false, "issue": "...", "lesson": "a short rule to remember"}"""


def critique(question: str, answer: str, facts: dict) -> dict:
    # A critic is only useful with something CHECKABLE: give it the raw facts, not just the answer.
    content = f"Question: {question}\n\nFacts from the merchant file: {facts}\n\nAnswer to check:
{answer}"
    r = client.messages.create(model=MODEL, max_tokens=1000, system=CRITIC_RUBRIC,
                               messages=[{"role": "user", "content": content}])
    text = "".join(b.text for b in r.content if b.type == "text")
    return json.loads(text[text.find("{"): text.rfind("}") + 1])


question = "Is ACME over its processing limit?"
tools_without_fx = [t for t in TOOLS if t["name"] != "get_fx_rate"]      # the planted weakness
weak_system = ("You are a merchant-risk analyst. Use the tools. Show your numbers, "
               "then finish with OVER_LIMIT / NEAR_LIMIT / OK.")

lessons = []
answer = run_agent(question, system=weak_system, tools=tools_without_fx)
print("FIRST ANSWER:", answer)

for attempt in range(2):                                    # never let the critic loop forever
    review = critique(question, answer, facts=MERCHANTS["ACME"])
    print(f"\nCRITIC: {review}")
    if review["ok"]:
        break
    lessons.append(review["lesson"])
    retry_prompt = question + "\n\nLessons from a reviewer:\n- " + "\n- ".join(lessons)
    answer = run_agent(retry_prompt, system=weak_system, tools=TOOLS)    # fix: give it the FX tool back
    print("\nREVISED ANSWER:", answer)

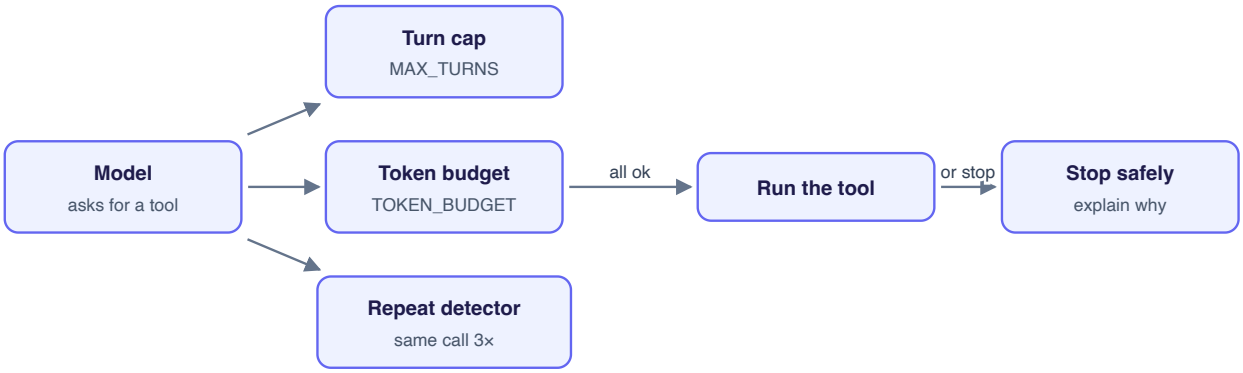
print("\nCorrect: 9.92M USD vs 9.5M USD limit -> OVER_LIMIT (104.4%)")
```

THEORY 8 MIN

Guardrails: stop runaway agents

An agent without limits is an unbounded loop that spends money.

Guardrails around the loop



1. The FX service is down and keeps saying 'please retry'. A naive loop retries forever, and spends money.
2. Guard 1: a hard cap on turns.
3. Guard 2: a token budget per run. Stop when it gets expensive.
4. Guard 3: if the same call fails 3 times, return a non-retryable error: 'do NOT retry'.
5. Only calls that pass every guard run. Otherwise the agent stops and says the data is unavailable.

Key points

- Every production loop needs: a turn cap, a token budget and repeat detection.
- Return errors the model can act on, including a clear "do NOT retry".

LAB INTERMEDIATE 15 MIN

Lab 4: Break it on purpose

Goal. Break the FX tool so it always says "please retry", then watch the guardrails stop the loop.

1. See how we break it

lab4_break_it.py: broken tool

```
def broken_fx(base: str, quote: str) -> float:
    raise ValueError("rate service temporarily unavailable, please retry")


FUNCTIONS["get_fx_rate"] = broken_fx      # break it on purpose
```

2. Find the three guards

Look for `# guard 1`, `# guard 2` and `# guard 3` in the loop.

lab4_break_it.py: guarded loop

```
for turn in range(1, MAX_TURNS + 1):                                    # guard 1
    response = client.messages.create(model=MODEL, max_tokens=4000, system=SYSTEM,
                                      tools=TOOLS, messages=messages)
    messages.append({"role": "assistant", "content": response.content})

    tokens_used += response.usage.input_tokens + response.usage.output_tokens
    if tokens_used > TOKEN_BUDGET:                                      # guard 2
        print(f"STOP: token budget exceeded ({tokens_used})")
        break

    if response.stop_reason != "tool_use":
        print("ANSWER:", "".join(b.text for b in response.content if b.type == "text"))
        break

    results = []
    for block in response.content:
        if block.type != "tool_use":
            continue
        key = block.name + json.dumps(block.input, sort_keys=True)
        seen_calls[key] = seen_calls.get(key, 0) + 1
        if seen_calls[key] > MAX_REPEATS:                               # guard 3
            output, is_error = "Error: this call keeps failing. Do NOT retry. Say the data is
unavailable.", True
        else:
            output, is_error = run_tool(block.name, block.input)
        print(f"[turn {turn}] {block.name}({block.input}) -> {output[:80]}")
        results.append({"type": "tool_result", "tool_use_id": block.id, "content": output, "is_error":
is_error})
    messages.append({"role": "user", "content": results})
else:
    print(f"STOP: hit MAX_TURNS={MAX_TURNS}")

print(f"\ntokens used: {tokens_used}")
```

3. Run it

RUN `python lab4_break_it.py`

4. Experiment

Set `MAX_REPEATS = 1`, then `TOKEN_BUDGET = 2000`. Which guard fires first?

Expected result. The FX call fails, the agent stops cleanly (data unavailable, or a guard fires) and prints the tokens used. No endless loop.

Full file

lab4_break_it.py

```
"""Day 2 – Lab 4: break the agent on purpose, then add guardrails.

The FX tool is broken: it always says "please retry". Without guardrails the agent could loop
(and spend money) forever. We add three guards:
  1. MAX_TURNS        – hard stop
  2. TOKEN_BUDGET     – stop when the run gets expensive
  3. repeat detector  – the same failing call 3 times -> tell the model to stop retrying

Run:  python lab4_break_it.py
"""
import json

from lab1_react_agent import FUNCTIONS, MODEL, SYSTEM, TOOLS, client, run_tool


def broken_fx(base: str, quote: str) -> float:
    raise ValueError("rate service temporarily unavailable, please retry")


FUNCTIONS["get_fx_rate"] = broken_fx      # break it on purpose

MAX_TURNS = 8
TOKEN_BUDGET = 50_000
MAX_REPEATS = 3

messages = [{"role": "user", "content": "Is ACME over its processing limit?"}]
tokens_used = 0
seen_calls = {}

for turn in range(1, MAX_TURNS + 1):                                        # guard 1
    response = client.messages.create(model=MODEL, max_tokens=4000, system=SYSTEM,
                                      tools=TOOLS, messages=messages)
    messages.append({"role": "assistant", "content": response.content})

    tokens_used += response.usage.input_tokens + response.usage.output_tokens
    if tokens_used > TOKEN_BUDGET:                                          # guard 2
        print(f"STOP: token budget exceeded ({tokens_used})")
        break

    if response.stop_reason != "tool_use":
        print("ANSWER:", "".join(b.text for b in response.content if b.type == "text"))
        break

    results = []
    for block in response.content:
        if block.type != "tool_use":
            continue
        key = block.name + json.dumps(block.input, sort_keys=True)
        seen_calls[key] = seen_calls.get(key, 0) + 1
        if seen_calls[key] > MAX_REPEATS:                                   # guard 3
            output, is_error = "Error: this call keeps failing. Do NOT retry. Say the data is
unavailable.", True
```

```
        else:
            output, is_error = run_tool(block.name, block.input)
        print(f"[turn {turn}] {block.name}({block.input}) -> {output[:80]}")
        results.append({"type": "tool_result", "tool_use_id": block.id, "content": output, "is_error":
is_error})
    messages.append({"role": "user", "content": results})
else:
    print(f"STOP: hit MAX_TURNS={MAX_TURNS}")

print(f"\ntokens used: {tokens_used}")
```

QUIZ 5 MIN

Check yourself

1. **How does your loop know the model wants to use a tool?**
   A. The text says 'I will use a tool'
   B. stop_reason == "tool_use" and there are tool_use blocks
   C. The response is empty
   D. usage.output_tokens is zero

2. **Why append the model's FULL reply to the history, not just its text?**
   A. To save tokens
   B. The tool_use ids (and thinking blocks) must be kept so the tool_result messages match up
   C. It's faster
   D. It isn't necessary

3. **In Lab 2, who marks a plan step as done?**
   A. The model
   B. The planner
   C. Your code
   D. The user

4. **When is a critic step worth its cost?**
   A. Always
   B. When there's a checkable rule and mistakes are expensive
   C. Only for long answers
   D. Never

5. **Which three guardrails should every agent loop have?**
   A. Temperature, top_p, top_k
   B. Turn cap, token budget, repeat detection
   C. Logging, retries, caching
   D. A bigger model, a longer prompt, more tools

RECAP 3 MIN

Day 2 recap

- An agent is a loop you own; the model only chooses the next action.
- Tool schemas are the real prompt: enums, clear descriptions, errors the model can recover from.
- Plan-and-Execute: code owns the checklist.
- Critics need a checkable rule and the facts, and a retry cap.
- Always ship guardrails: turn cap, token budget, repeat detection.

Next: Day 3: the same agent in LangChain. You'll point at exactly which of today's lines the framework replaces.

Quiz answer key

1. **B. stop_reason == "tool_use" and there are tool_use blocks**
   stop_reason and the tool_use blocks are the machine-readable signal.
2. **B. The tool_use ids (and thinking blocks) must be kept so the tool_result messages match up**
   Each tool_result references a tool_use id from the previous reply. History is append-only.
3. **C. Your code**
   Code owns the plan state. That's what makes progress reliable and auditable.
4. **B. When there's a checkable rule and mistakes are expensive**
   A critic needs a real signal to check against: rules plus facts.
5. **B. Turn cap, token budget, repeat detection**
   These stop runaway loops and runaway bills.